

R Programming Practice (Part 2)

Tim Irwin

2026-04-16

R Programming Practice (Part 2)

This document is for practice based on *R Programming for Data Science* by Roger D. Peng (pages 12–22 and 35–43):

1. R Nuts and Bolts
2. Subsetting R Objects
3. Vectorized Operations

Write your answers inside the R chunks below.

You may want to start with:

```
setwd("/Users/timothyirwin/Desktop/Coursera/Foundations/data science/Practice")
rm(list = ls())
```

1. R Nuts and Bolts

Entering Input and Evaluation

Exercise 1.1

Assign the value 42 to an object called `x`, then print it using auto-printing (just type the object name).

```
x <- 42
```

Exercise 1.2

Assign the string "hello world" to an object called `msg`. Then use the `print()` function to explicitly print it.

```
msg <- c("hello world")
print(msg)
```

```
## [1] "hello world"
```

Exercise 1.3

Create a vector `x <- 10:30` and print it. Notice the index numbers in square brackets on the left side of the output. What does `[13]` on the second line represent?

```
x <- 10:30
x
```

```
## [1] 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

```
# the thirteen number?? it doesn't appear on my output
```

R Objects

Exercise 1.4

What are the five basic (“atomic”) classes of objects in R? Assign one example of each to a different variable (e.g., a, b, c_val, d, e), and use `class()` on each to confirm.

```
a <- c("x", "y", "z") # character
b <- c(1, 4, 7, 9)    # numeric
c_val <- 4:27          # integer
d <- c(1+0i, 2+4i)    # complex
e <- c(T, F, F, F, T) # logical
```

```
class(a)
```

```
## [1] "character"
```

```
class(b)
```

```
## [1] "numeric"
```

```
class(c_val)
```

```
## [1] "integer"
```

```
class(d)
```

```
## [1] "complex"
```

```
class(e)
```

```
## [1] "logical"
```

```
# what's a more efficient way to run the classes rather than having to enter  
# individually? Loop??
```

Exercise 1.5

What is the one rule about vectors in R? What is the exception to this rule? Write your answer as a comment.

```
# they all have to be of the same class, except for lists
```

Numbers

Exercise 1.6

What is the difference between 1 and 1L in R? Use `class()` to show the difference.

```
class(1)
```

```
## [1] "numeric"
```

```
class(1L)
```

```
## [1] "integer"
```

```
# so the L makes it an integer? so i can add L to any whole number and make it  
# an integer? what would the point of doing so be?
```

Exercise 1.7

What does `1 / 0` return in R? What about `0 / 0`? Try both.

```
# Your code here
```

Attributes

Exercise 1.8

Create a vector `y <- 1:6`. Use `attributes()` to check its attributes. Then convert it to a matrix with `dim(y1) <- c(2, 3)` and check `attributes()` again. What changed?

```
y <- 1:6
attributes(y)
```

```
## NULL
```

```
dim(y) <- c(2, 3)
attributes(y)
```

```
## $dim
## [1] 2 3
```

```
# i want to learn more about creating matrices in the next practice round
```

Creating Vectors

Exercise 1.9

Use `c()` to create each of the following vectors:

- A numeric vector: 0.5, 0.6
- A logical vector: TRUE, FALSE
- A character vector: "a", "b", "c"
- An integer sequence from 9 to 29

```
num <- c(0.5, 0.6)
num
```

```
## [1] 0.5 0.6
```

```
lo <- c(TRUE, FALSE)
lo
```

```
## [1] TRUE FALSE
```

```
char <- c("a", "b", "c")
char
```

```
## [1] "a" "b" "c"
```

```
int <- 9:29
int
```

```
## [1] 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
```

Exercise 1.10

Use `vector()` to create a numeric vector of length 10. What values does it contain by default?

```
vector("numeric", length = 10)
```

```
## [1] 0 0 0 0 0 0 0 0 0 0
# contains 0 by default
```

Mixing Objects and Coercion

Exercise 1.11

What happens when you run each of the following? Predict the class before running, then check with `class()`.

```
y1 <- c(1.7, "a") # character
y2 <- c(TRUE, 2) # numeric
y3 <- c("a", TRUE) # character
```

```
# Check classes here
class(y1)
```

```
## [1] "character"
class(y2)
```

```
## [1] "numeric"
class(y3)
```

```
## [1] "character"
```

Exercise 1.12

Create a vector `x <- 0:6`. Use `as.numeric()`, `as.logical()`, and `as.character()` to explicitly coerce it. What does each produce?

```
x1 <- 0:6
as.numeric(x1)
```

```
## [1] 0 1 2 3 4 5 6
as.logical(x1)
```

```
## [1] FALSE TRUE TRUE TRUE TRUE TRUE TRUE
as.character(x1)
```

```
## [1] "0" "1" "2" "3" "4" "5" "6"
```

Exercise 1.13

Try coercing the character vector `c("a", "b", "c")` to numeric. What happens?

```
g <- c("a", "b", "c")
as.numeric(g)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA NA NA
```

```
# NA values introduced by coercion
```

Matrices

Exercise 1.14

Create a 2 x 3 matrix using `matrix(nrow = 2, ncol = 3)` without specifying data. What are the default values?

```
matrix(nrow = 2, ncol = 3)
```

```
##      [,1] [,2] [,3]
## [1,]  NA  NA  NA
## [2,]  NA  NA  NA
# default values are NA
```

Exercise 1.15

Create a matrix with the values 1 through 6, with 2 rows and 3 columns. Are the values filled by column or by row?

```
matrix(1:6, nrow = 2, ncol = 3)
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
# filled by column
```

Exercise 1.16

Create a vector `m <- 1:10`. Then add a dimension attribute to turn it into a 2 x 5 matrix using `dim(m) <- c(2, 5)`.

```
m <- 1:10
dim(m) <- c(2, 5)
m
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
```

```
# next time don't give me the code to do this
```

Exercise 1.17

Create two vectors, `x <- 1:3` and `y <- 10:12`. Use `cbind()` to combine them as columns, and `rbind()` to combine them as rows.

```
x4 <- 1:3
y4 <- 10:12
cbind(x4, y4)
```

```
##      x4 y4
## [1,]  1 10
## [2,]  2 11
## [3,]  3 12
```

```
rbind(x4, y4)
```

```
##      [,1] [,2] [,3]
## x4      1    2    3
```

```
## y4    10    11    12
```

Lists

Exercise 1.18

Create a list containing: the number 1, the string "a", the value TRUE, and the complex number 1+4i. Print it and notice the double-bracket `[[ ]` indexing in the output.

```
list(1, "a", TRUE, 1+4i)
```

```
## [[1]]
## [1] 1
##
## [[2]]
## [1] "a"
##
## [[3]]
## [1] TRUE
##
## [[4]]
## [1] 1+4i
```

Exercise 1.19

Create an empty list of length 5 using `vector("list", length = 5)`. What does each element contain?

```
vector("list", length = 5)
```

```
## [[1]]
## NULL
##
## [[2]]
## NULL
##
## [[3]]
## NULL
##
## [[4]]
## NULL
##
## [[5]]
## NULL
```

```
# contains NULL values
```

Factors

Exercise 1.20

Create a factor from the vector `c("yes", "yes", "no", "yes", "no")`. Use `table()` to count the levels, and `unclass()` to see the underlying integer representation.

```
k <- factor(c("yes", "yes", "no", "yes", "no"))
table(k)
```

```
## k
##  no yes
```

```
## 2 3
```

```
unclass(k)
```

```
## [1] 2 2 1 2 1
## attr("levels")
## [1] "no" "yes"
```

Exercise 1.21

Create the same factor as above, but this time set the levels so that "yes" comes before "no". Hint: use the `levels` argument.

```
k <- factor(c("yes", "yes", "no", "yes", "no"),
            levels = c("yes", "no"))
k
```

```
## [1] yes yes no yes no
## Levels: yes no
```

Missing Values

Exercise 1.22

Create a vector `x <- c(1, 2, NA, 10, 3)`. Use `is.na()` to identify the missing values. Then use `is.nan()` on the same vector. Are the results the same?

```
x5 <- c(1, 2, NA, 10, 3)
is.na(x5)
```

```
## [1] FALSE FALSE TRUE FALSE FALSE
```

```
is.nan(x5)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE
```

Exercise 1.23

Create a vector `x <- c(1, 2, NaN, NA, 4)`. Apply both `is.na()` and `is.nan()` to it. What is the difference between how they treat `NaN` and `NA`?

```
x6 <- c(1, 2, NaN, NA, 4)
is.na(x6)
```

```
## [1] FALSE FALSE TRUE TRUE FALSE
```

```
is.nan(x6)
```

```
## [1] FALSE FALSE TRUE FALSE FALSE
```

```
# is.na() gives a TRUE logical value for NAs and NaN
# whereas is.nan() gives a TRUE logical value for only NaN
```

Data Frames

Exercise 1.24

Create a data frame with two columns: `foo` containing `1:4` and `bar` containing `c(TRUE, TRUE, FALSE, FALSE)`. Use `nrow()` and `ncol()` to check its dimensions.

```
z <- data.frame(foo = 1:4, bar = c(TRUE, TRUE, FALSE, FALSE))
nrow(z)
```

```
## [1] 4
```

```
ncol(z)
```

```
## [1] 2
```

Exercise 1.25

What is the difference between a data frame and a matrix? Write your answer as a comment.

```
# data frames can store different types of objects in each column, whereas  
# matrices have to have the same class for all elements
```

Names

Exercise 1.26

Create a vector `x <- 1:3`. Assign the names "New York", "Seattle", and "Los Angeles" using the `names()` function. Print `x` and its names.

```
x7 <- 1:3  
names(x7) <- c("New York", "Seattle", "Los Angeles")  
x7
```

```
##      New York      Seattle Los Angeles  
##          1          2          3
```

Exercise 1.27

Create a named list: `x <- list("Los Angeles" = 1, Boston = 2, London = 3)`. Use `names()` to extract the names.

```
x8 <- list("Los Angeles" = 1, "Boston" = 2, "London" = 3)  
names(x8)
```

```
## [1] "Los Angeles" "Boston"      "London"
```

Exercise 1.28

Create a 2 x 2 matrix from 1:4. Assign row names "a" and "b", and column names "c" and "d" using `dimnames()`.

```
m <- matrix(1:4, nrow = 2, ncol = 2)  
dimnames(m) <- list(c("a", "b"), c("c", "d"))  
m
```

```
##   c d  
## a 1 3  
## b 2 4
```

2. Subsetting R Objects

Subsetting a Vector

Exercise 2.1

Create a vector `x <- c("a", "b", "c", "c", "d", "a")`. Extract the first element, the second element, and elements 1 through 4.


```
x8 <- c("a", "b", "c", "c", "d", "a")
x8[1]
```

```
## [1] "a"
```

```
x8[2]
```

```
## [1] "b"
```

```
x8[1:4]
```

```
## [1] "a" "b" "c" "c"
```

Exercise 2.2

Using the same vector `x`, extract elements 1, 3, and 4 using `c()` to specify the indices.

```
x8[c(1, 3, 4)]
```

```
## [1] "a" "c" "c"
```

Exercise 2.3

Using the same vector `x`, extract all elements that come after "a" in alphabetical order. Do this in two ways: (1) by first creating a logical vector, and (2) by subsetting directly with a logical expression.

```
u <- x8 > "a"
u
```

```
## [1] FALSE TRUE TRUE TRUE TRUE FALSE
```

```
x8[x8 > "a"]
```

```
## [1] "b" "c" "c" "d"
```

Subsetting a Matrix

Exercise 2.4

Create a 2 x 3 matrix: `x <- matrix(1:6, 2, 3)`. Extract the element in row 1, column 2. Then extract the element in row 2, column 1.

```
x9 <- matrix(1:6, 2, 3)
x9[2, 1]
```

```
## [1] 2
```

Exercise 2.5

Using the same matrix, extract the entire first row. Then extract the entire second column.

```
x9[1, ]
```

```
## [1] 1 3 5
```

Exercise 2.6

When you extract `x[1, 2]`, R returns a vector of length 1, not a 1x1 matrix. How do you preserve the matrix structure? Try it with `drop = FALSE`.

```
x9[1, 2, drop = FALSE]
```

```
##      [,1]
## [1,]    3
```

Exercise 2.7

Extract the first row of the matrix using `x[1, ]`. What is the result's class? Now try `x[1, , drop = FALSE]`. What class is the result now?

```
# Your code here
```

```
u1 <- x9[1, ]
class(u1)
```

```
## [1] "integer"
```

```
u2 <- x9[1, , drop = FALSE]
class(u2)
```

```
## [1] "matrix" "array"
```

Subsetting Lists

Exercise 2.8

Create a list: `x <- list(foo = 1:4, bar = 0.6)`. Extract the first element using `[[`. Then extract `bar` using both `["bar"]` and `$bar`.

```
x10 <- list(foo = 1:4, bar = 0.6)
x10[[1]]
```

```
## [1] 1 2 3 4
```

```
x10[["bar"]]
```

```
## [1] 0.6
```

```
x10$bar
```

```
## [1] 0.6
```

Exercise 2.9

Create a list: `x <- list(foo = 1:4, bar = 0.6, baz = "hello")`. Store "foo" in a variable called `name`. Use `x[[name]]` to extract the element. Why does `x$name` return `NULL` instead?

```
x11 <- list(foo = 1:4, bar = 0.6, baz = "hello")
name <- "foo"
x11[[name]]
```

```
## [1] 1 2 3 4
```

```
x11$name
```

```
## NULL
```

```
# the $ can only be used with literal names
```

Subsetting Nested Elements

Exercise 2.10

Create a nested list: `x <- list(a = list(10, 12, 14), b = c(3.14, 2.81))`. Extract the third element of the first element in two ways: using `x[[c(1, 3)]]` and using `x[[1]][[3]]`.

```
x12 <- list(a = list(10, 12, 14), b = c(3.14, 2.81))
x12[[c(1, 3)]]
```

```
## [1] 14
```

```
x12[[1]][[3]]
```

```
## [1] 14
```

Extracting Multiple Elements

Exercise 2.11

Using `x <- list(foo = 1:4, bar = 0.6, baz = "hello")`, extract the first and third elements using `[`. What class is the result? How is this different from using `[[`?

```
x13 <- list(foo = 1:4, bar = 0.6, baz = "hello")
x13[c(1, 3)]
```

```
## $foo
```

```
## [1] 1 2 3 4
```

```
##
```

```
## $baz
```

```
## [1] "hello"
```

```
x13[[c(1, 3)]]
```

```
## [1] 3
```

```
# the [] returns a list, not really sure why
```

Partial Matching

Exercise 2.12

Create a list `x <- list(aardvark = 1:5)`. Try `x$a`, `x[["a"]]`, and `x[["a", exact = FALSE]]`. What does each return?

```
x14 <- list(aardvark = 1:5)
x14$a
```

```
## [1] 1 2 3 4 5
```

```
x14[["a"]] # this does NULL because it is looking for only "a" whereas
```

```
## NULL
```

```
# the other ones are looking for a value that starts with a
```

```
x14[["a", exact = FALSE]]
```

```
## [1] 1 2 3 4 5
```

Removing NA Values

Exercise 2.13

Create a vector `x <- c(1, 2, NA, 4, NA, 5)`. Use `is.na()` and logical negation (`!`) to extract only the non-missing values.

```
x15 <- c(1, 2, NA, 4, NA, 5)
cod <- is.na(x15)
x15[!cod]
```

```
## [1] 1 2 4 5
```

Exercise 2.14

Create two vectors with NAs in the same positions:

- `x <- c(1, 2, NA, 4, NA, 5)`
- `y <- c("a", "b", NA, "d", NA, "f")`

Use `complete.cases()` to find which positions have no missing values in either vector, then subset both `x` and `y` using that result.

```
x15 <- c(1, 2, NA, 4, NA, 5)
y5 <- c("a", "b", NA, "d", NA, "f")

good <- complete.cases(x15, y5)
good
```

```
## [1] TRUE TRUE FALSE TRUE FALSE TRUE
```

Exercise 2.15

Load the built-in `airquality` dataset. Use `complete.cases()` to create a version with no missing values. How many complete rows are there compared to the original?

```
good1 <- complete.cases(airquality)
airquality[good1, ]
```

```
##      Ozone Solar.R Wind Temp Month Day
## 1      41      190  7.4   67     5   1
## 2      36      118  8.0   72     5   2
## 3      12      149 12.6   74     5   3
## 4      18      313 11.5   62     5   4
## 7      23      299  8.6   65     5   7
## 8      19       99 13.8   59     5   8
## 9       8       19 20.1   61     5   9
## 12     16      256  9.7   69     5  12
## 13     11      290  9.2   66     5  13
## 14     14      274 10.9   68     5  14
## 15     18       65 13.2   58     5  15
## 16     14      334 11.5   64     5  16
## 17     34      307 12.0   66     5  17
## 18      6       78 18.4   57     5  18
## 19     30      322 11.5   68     5  19
## 20     11       44  9.7   62     5  20
## 21      1        8  9.7   59     5  21
## 22     11      320 16.6   73     5  22
## 23      4       25  9.7   61     5  23
## 24     32       92 12.0   61     5  24
## 28     23       13 12.0   67     5  28
## 29     45      252 14.9   81     5  29
## 30    115      223  5.7   79     5  30
## 31     37      279  7.4   76     5  31
```

## 38	29	127	9.7	82	6	7
## 40	71	291	13.8	90	6	9
## 41	39	323	11.5	87	6	10
## 44	23	148	8.0	82	6	13
## 47	21	191	14.9	77	6	16
## 48	37	284	20.7	72	6	17
## 49	20	37	9.2	65	6	18
## 50	12	120	11.5	73	6	19
## 51	13	137	10.3	76	6	20
## 62	135	269	4.1	84	7	1
## 63	49	248	9.2	85	7	2
## 64	32	236	9.2	81	7	3
## 66	64	175	4.6	83	7	5
## 67	40	314	10.9	83	7	6
## 68	77	276	5.1	88	7	7
## 69	97	267	6.3	92	7	8
## 70	97	272	5.7	92	7	9
## 71	85	175	7.4	89	7	10
## 73	10	264	14.3	73	7	12
## 74	27	175	14.9	81	7	13
## 76	7	48	14.3	80	7	15
## 77	48	260	6.9	81	7	16
## 78	35	274	10.3	82	7	17
## 79	61	285	6.3	84	7	18
## 80	79	187	5.1	87	7	19
## 81	63	220	11.5	85	7	20
## 82	16	7	6.9	74	7	21
## 85	80	294	8.6	86	7	24
## 86	108	223	8.0	85	7	25
## 87	20	81	8.6	82	7	26
## 88	52	82	12.0	86	7	27
## 89	82	213	7.4	88	7	28
## 90	50	275	7.4	86	7	29
## 91	64	253	7.4	83	7	30
## 92	59	254	9.2	81	7	31
## 93	39	83	6.9	81	8	1
## 94	9	24	13.8	81	8	2
## 95	16	77	7.4	82	8	3
## 99	122	255	4.0	89	8	7
## 100	89	229	10.3	90	8	8
## 101	110	207	8.0	90	8	9
## 104	44	192	11.5	86	8	12
## 105	28	273	11.5	82	8	13
## 106	65	157	9.7	80	8	14
## 108	22	71	10.3	77	8	16
## 109	59	51	6.3	79	8	17
## 110	23	115	7.4	76	8	18
## 111	31	244	10.9	78	8	19
## 112	44	190	10.3	78	8	20
## 113	21	259	15.5	77	8	21
## 114	9	36	14.3	72	8	22
## 116	45	212	9.7	79	8	24
## 117	168	238	3.4	81	8	25
## 118	73	215	8.0	86	8	26

```
## 120    76    203  9.7  97    8  28
## 121   118    225  2.3  94    8  29
## 122    84    237  6.3  96    8  30
## 123    85    188  6.3  94    8  31
## 124    96    167  6.9  91    9   1
## 125    78    197  5.1  92    9   2
## 126    73    183  2.8  93    9   3
## 127    91    189  4.6  93    9   4
## 128    47     95  7.4  87    9   5
## 129    32     92 15.5  84    9   6
## 130    20    252 10.9  80    9   7
## 131    23    220 10.3  78    9   8
## 132    21    230 10.9  75    9   9
## 133    24    259  9.7  73    9  10
## 134    44    236 14.9  81    9  11
## 135    21    259 15.5  76    9  12
## 136    28    238  6.3  77    9  13
## 137     9     24 10.9  71    9  14
## 138    13    112 11.5  71    9  15
## 139    46    237  6.9  78    9  16
## 140    18    224 13.8  67    9  17
## 141    13     27 10.3  76    9  18
## 142    24    238 10.3  68    9  19
## 143    16    201  8.0  82    9  20
## 144    13    238 12.6  64    9  21
## 145    23     14  9.2  71    9  22
## 146    36    139 10.3  81    9  23
## 147     7     49 10.3  69    9  24
## 148    14     20 16.6  63    9  25
## 149    30    193  6.9  70    9  26
## 151    14    191 14.3  75    9  28
## 152    18    131  8.0  76    9  29
## 153    20    223 11.5  68    9  30
```

```
# idk if i did this right
```

3. Vectorized Operations

Exercise 3.1

Create two vectors: `x <- 1:4` and `y <- 6:9`. Add them together. Then subtract, multiply, and divide them element-wise.

```
x15 <- 1:4
y15 <- 6:9
```

```
x15 + y15
```

```
## [1]  7  9 11 13
```

```
x15 - y15
```

```
## [1] -5 -5 -5 -5
```

```
x15 * y15

## [1]  6 14 24 36

x15 / y15

## [1] 0.1666667 0.2857143 0.3750000 0.4444444
```

Exercise 3.2

Using `x <- 1:4`, test which elements are greater than 2, which are greater than or equal to 2, and which are less than 3. What class of vector does each comparison return?

```
x16 <- 1:4
x16 > 2

## [1] FALSE FALSE  TRUE  TRUE

x16 >= 2

## [1] FALSE  TRUE  TRUE  TRUE

x16 < 3

## [1]  TRUE  TRUE FALSE FALSE
```

Exercise 3.3

Create two 2x2 matrices:

- `x <- matrix(1:4, 2, 2)`
- `y <- matrix(rep(10, 4), 2, 2)`

Perform element-wise multiplication (`*`), element-wise division (`/`), and true matrix multiplication (`%*%`). How do the results differ?

```
x17 <- matrix(1:4, 2, 2)
y17 <- matrix(rep(10, 4), 2, 2)
x17 * y17

##      [,1] [,2]
## [1,]   10   30
## [2,]   20   40

x17 / y17

##      [,1] [,2]
## [1,]  0.1  0.3
## [2,]  0.2  0.4

x17 %*% y17

##      [,1] [,2]
## [1,]   40   40
## [2,]   60   60
```

Mini Challenges

Challenge 1

Create a factor from `c("low", "med", "high", "low", "high")` with levels ordered as low, med, high. Verify the ordering with `levels()`.

```
factor1 <- factor(c("low", "med", "high", "low", "high"),
                  levels = c("low", "med", "high"))
factor1
```

```
## [1] low med high low high
## Levels: low med high
```

can you explain to me when i need to use the factor function and when i don't

Challenge 2

Create a named list with three elements: `name = "Alice"`, `scores = c(90, 85, 92)`, and `passed = TRUE`. Extract the second score using nested indexing.

```
list1 <- list(name = "Alice", scores = c(90, 85, 92), passed = TRUE)
list1[[c(2, 2)]]
```

```
## [1] 85
```

Challenge 3

Create a 3x3 matrix of the numbers 1 through 9. Extract the submatrix consisting of rows 1-2 and columns 2-3. Use `drop = FALSE` to keep it as a matrix.

```
matrix1 <- matrix(1:9, nrow = 3, ncol = 3)
matrix1
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
matrix1[1:2, 2:3, drop = FALSE]
```

```
##      [,1] [,2]
## [1,]    4    7
## [2,]    5    8
```

Challenge 4

Create a vector with some NAs: `z <- c(5, NA, 8, NA, 12, 3, NA)`. In one line, calculate the sum of the non-missing values.

```
z1 <- c(5, NA, 8, NA, 12, 3, NA)
sum(z1, na.rm = TRUE)
```

```
## [1] 28
```

Challenge 5

Given `x <- c(10, 20, 30, 40, 50)` and `y <- c(1, 0, 1, 0, 1)`, use vectorized operations to extract only the elements of `x` where `y` equals 1.


```
x17 <- c(10, 20, 30, 40, 50)
y17 <- c(1, 0, 1, 0, 1)
x17[y17 == 1]
```

```
## [1] 10 30 50
```